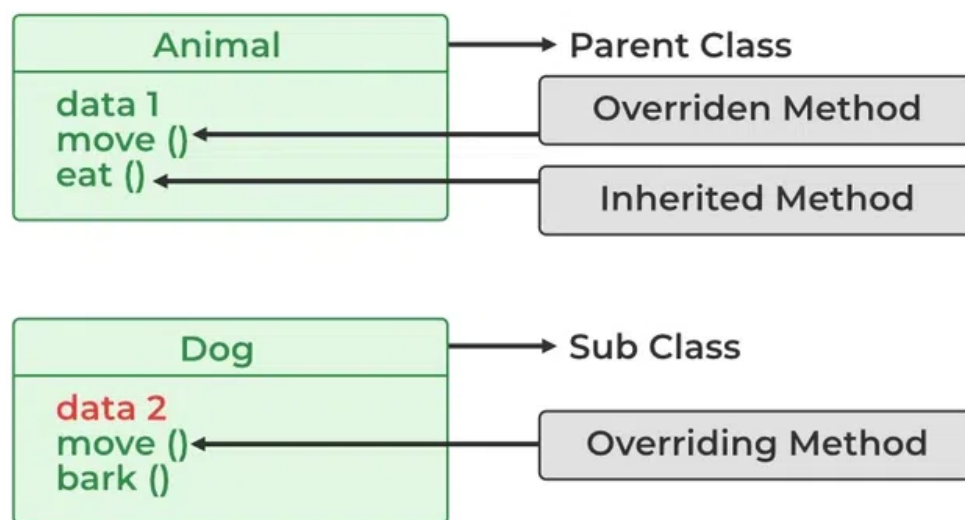


Overriding in Java

Last Updated : 04 Oct, 2024



In Java, Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a subclass has the same name, the same parameters or signature, and the same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to *override* the method in the super-class.



Method overriding is one of the ways by which [Java achieves Run Time Polymorphism](#). The version of a method that is executed will be determined by the object that is used to invoke it. If an object of a parent class is used to invoke the method, then the version in the parent class will be executed, but if an object of the subclass is used to invoke the method, then the version in the child class will be executed. In other words, *it is the type of the object being referred to* (not the type of the reference variable) that determines which version of an overridden method will be executed.

Example of Method Overriding in Java

Below is the implementation of the Java Method Overriding:

```
Java
// Implementation of Method Overriding in Java
// ... (code content) ...
```



```
1 // Java program to demonstrate
2 // method overriding in java
3
4 // Base Class
5 class Parent {
6     void show() { System.out.println("Parent's show()");
7 }
8
9 // Inherited class
10 class Child extends Parent {
11     // This method overrides show() of Parent
12     @Override void show()
13     {
14         System.out.println("Child's show()");
15     }
16 }
17
18 // Driver class
19 class Main {
20     public static void main(String[] args)
21     {
22         // If a Parent type reference refers
23         // to a Parent object, then Parent's
24         // show is called
25         Parent obj1 = new Parent();
26         obj1.show();
27
28         // If a Parent type reference refers
29         // to a Child object Child's show()
30         // is called. This is called RUN TIME
31         // POLYMORPHISM.
32         Parent obj2 = new Child();
33         obj2.show();
34     }
35 }
```

Output

Parent's show()

Child's show()

For more complex examples of how to implement overriding in Java, the [Java Programming Course](#) provides detailed exercises and real-world scenarios to help deepen your understanding.

Rules for Java Method Overriding

1. Overriding and Access Modifiers

The [access modifier](#) for an overriding method can allow more, but not less, access than the overridden method. For example, a protected instance method in the superclass can be made public, but not private, in the subclass. Doing so will generate a **compile-time error**.

Java

```
1 // A Simple Java program to demonstrate
2 // Overriding and Access-Modifiers
3
4 class Parent {
5     // private methods are not overridden
6     private void m1()
7     {
8         System.out.println("From parent m1()");
9     }
10
11     protected void m2()
12     {
13         System.out.println("From parent m2()");
14     }
15 }
16
17 class Child extends Parent {
18     // new m1() method
19     // unique to Child class
20     private void m1()
21     {
```

```

22         System.out.println("From child m1()");
23     }
24
25     // overriding method
26     // with more accessibility
27     @Override public void m2()
28     {
29         System.out.println("From child m2()");
30     }
31 }
32
33 // Driver class
34 class Main {
35     public static void main(String[] args)
36     {
37         Parent obj1 = new Parent();
38         obj1.m2();
39         Parent obj2 = new Child();
40         obj2.m2();
41     }
42 }

```

Output

```

From parent m2()
From child m2()

```

2. Final methods can not be overridden

If we don't want a method to be overridden, we declare it as [final](#). Please see [Using Final with Inheritance](#).

Java



```

1 // A Java program to demonstrate that
2 // final methods cannot be overridden
3
4 class Parent {

```

```

5      // Can't be overridden
6      final void show() {}
7  }
8
9  class Child extends Parent {
10     // This would produce error
11     void show() {}
12 }

```

Output

```

13: error: show() in Child cannot override show() in Parent
    void show() {  }
        ^
    overridden method is final

```

3. Static methods can not be overridden (Method Overriding vs Method Hiding):

When you define a static method with the same signature as a static method in the base class, it is known as method hiding. The following table summarizes what happens when you define a method with the same signature as a method in a super-class.

	Superclass Instance Method	Superclass Static Method
Subclass Instance Method	Overrides	Generates a compile-time error
Subclass Static Method	Generates a compile-time error	Hides

Java



```
1 // Java program to show that
2 // if the static method is redefined by
3 // a derived class, then it is not
4 // overriding, it is hiding
5
6 class Parent {
7     // Static method in base class
8     // which will be hidden in subclass
9     static void m1()
10    {
11        System.out.println("From parent "
12                           + "static m1()");
13    }
14
15    // Non-static method which will
16    // be overridden in derived class
17    void m2()
18    {
19        System.out.println(
20            "From parent "
21            + "non - static(instance) m2() ");
22    }
23 }
24
25 class Child extends Parent {
26     // This method hides m1() in Parent
27     static void m1()
28     {
29         System.out.println("From child static m1()");
30     }
31
32     // This method overrides m2() in Parent
33     @Override public void m2()
34     {
35         System.out.println(
36             "From child "
37             + "non - static(instance) m2() ");
38     }
39 }
40
```

```

41 // Driver class
42 class Main {
43     public static void main(String[] args)
44     {
45         Parent obj1 = new Child();
46
47         // As per overriding rules this
48         // should call to class Child static
49         // overridden method. Since static
50         // method can not be overridden, it
51         // calls Parent's m1()
52         obj1.m1();
53
54         // Here overriding works
55         // and Child's m2() is called
56         obj1.m2();
57     }
58 }

```

Output

```

From parent static m1()
From child non - static(instance) m2()

```

4. Private methods can not be overridden

Private methods cannot be overridden as they are **bonded during compile time**. Therefore we can't even override private methods in a subclass. (See [this](#) for details).

Java

```

1 class SuperClass {
2     private void privateMethod()
3     {
4         System.out.println(
5             "This is a private method in SuperClass");
6     }

```

```

7
8     public void publicMethod()
9     {
10         System.out.println(
11             "This is a public method in SuperClass");
12         privateMethod();
13     }
14 }
15
16 class SubClass extends SuperClass {
17     // This is a new method with the same name as the
18     // private method in SuperClass
19     private void privateMethod()
20     {
21         System.out.println(
22             "This is a private method in SubClass");
23     }
24
25     // This method overrides the public method in
    SuperClass
26     public void publicMethod()
27     {
28         System.out.println(
29             "This is a public method in SubClass");
30         privateMethod(); // calls the private method in
31                           // SubClass, not SuperClass
32     }
33 }
34
35 public class Test {
36     public static void main(String[] args)
37     {
38         SuperClass obj1 = new SuperClass();
39         obj1.publicMethod(); // calls the public method
    in
40                               // SuperClass
41
42         SubClass obj2 = new SubClass();
43         obj2.publicMethod(); // calls the overridden
    public
44                               // method in SubClass
45     }

```

Output

```
This is a public method in SuperClass
This is a private method in SuperClass
This is a public method in SubClass
This is a private method in SubClass
```

5. The overriding method must have the same return type (or subtype)

From Java 5.0 onwards it is possible to have different return types for an overriding method in the child class, but the child's return type should be a sub-type of the parent's return type. This phenomenon is known as the **covariant return type**.

Java

```
1  class SuperClass {
2      public Object method()
3      {
4          System.out.println(
5              "This is the method in SuperClass");
6          return new Object();
7      }
8  }
9
10 class SubClass extends SuperClass {
11     public String method()
12     {
13         System.out.println(
14             "This is the method in SubClass");
15         return "Hello, World!";
16     }
17 }
18
19 public class Test {
20     public static void main(String[] args)
21     {
```

```

22         SuperClass obj1 = new SuperClass();
23         obj1.method();
24
25         SubClass obj2 = new SubClass();
26         obj2.method();
27     }
28 }

```

Output

```

This is the method in SuperClass
This is the method in SubClass

```

6. Invoking overridden method from sub-class

We can call the parent class method in the overriding method using the super keyword.

Java

```

1  // A Java program to demonstrate that overridden
2  // method can be called from sub-class
3
4  // Base Class
5  class Parent {
6      void show() { System.out.println("Parent's show()");
7  }
8
9  // Inherited class
10 class Child extends Parent {
11     // This method overrides show() of Parent
12     @Override void show()
13     {
14         super.show();
15         System.out.println("Child's show()");
16     }
17 }

```

```
18
19 // Driver class
20 class Main {
21     public static void main(String[] args)
22     {
23         Parent obj = new Child();
24         obj.show();
25     }
26 }
```

Output

```
Parent's show()
Child's show()
```

Overriding and Constructor

We **can not override the constructor** as the parent and child class can never have a constructor with the same name(The constructor name must always be the same as the Class name).

Overriding and Exception-Handling

Below are two rules to note when overriding methods related to exception handling.

Rule #1

If the super-class overridden method does not throw an exception, the subclass overriding method can only throw **the unchecked exception**, throwing a checked exception will lead to a compile-time error.

Java



```
1 // Java program to demonstrate overriding when
2 // superclass method does not declare an exception
3
4 class Parent {
```

```

5      void m1() { System.out.println("From parent m1()");
6      }
7      void m2() { System.out.println("From parent m2()");
8      }
9
10     class Child extends Parent {
11         @Override
12         // no issue while throwing unchecked exception
13         void m1() throws ArithmeticException
14         {
15             System.out.println("From child m1()");
16         }
17
18         @Override
19         // compile-time error
20         // issue while throwing checked exception
21         void m2() throws Exception
22         {
23             System.out.println("From child m2");
24         }
25     }

```

Output

```

error: m2() in Child cannot override m2() in Parent
    void m2() throws Exception{ System.out.println("From child m2");}
        ^
    overridden method does not throw Exception

```

Rule #2

If the superclass overridden method does throw an exception, the subclass overriding method can only throw the same, subclass exception. Throwing parent exceptions in [the Exception hierarchy](#) will lead to compile time error. Also, there is no issue if the subclass overridden method is not throwing any exception.



```
1 // Java program to demonstrate overriding when
2 // superclass method does declare an exception
3
4 class Parent {
5     void m1() throws RuntimeException
6     {
7         System.out.println("From parent m1()");
8     }
9 }
10
11 class Child1 extends Parent {
12     @Override
13     // no issue while throwing same exception
14     void m1() throws RuntimeException
15     {
16         System.out.println("From child1 m1()");
17     }
18 }
19 class Child2 extends Parent {
20     @Override
21     // no issue while throwing subclass exception
22     void m1() throws ArithmeticException
23     {
24         System.out.println("From child2 m1()");
25     }
26 }
27 class Child3 extends Parent {
28     @Override
29     // no issue while not throwing any exception
30     void m1()
31     {
32         System.out.println("From child3 m1()");
33     }
34 }
35 class Child4 extends Parent {
36     @Override
37     // compile-time error
38     // issue while throwing parent exception
39     void m1() throws Exception
```

```

40     {
41         System.out.println("From child4 m1()");
42     }
43 }

```

Output

```

error: m1() in Child4 cannot override m1() in Parent
    void m1() throws Exception
        ^
    overridden method does not throw Exception

```

Overriding and Abstract Method

Abstract methods in an interface or abstract class **are meant to be overridden** in derived concrete classes otherwise a compile-time error will be thrown.

Overriding and **Synchronized/strictfp** Method

The presence of a synchronized/strictfp modifier with the method has no effect on the rules of overriding, i.e. it's possible that a synchronized/strictfp method can override a non-synchronized/strictfp one and vice-versa.

Note:

1. In C++, we **need virtual keyword to** achieve overriding or Run Time Polymorphism. In Java, methods are virtual by default.
2. We can have multilevel method-overriding.

Java

```

1 // A Java program to demonstrate
2 // multi-level overriding
3

```

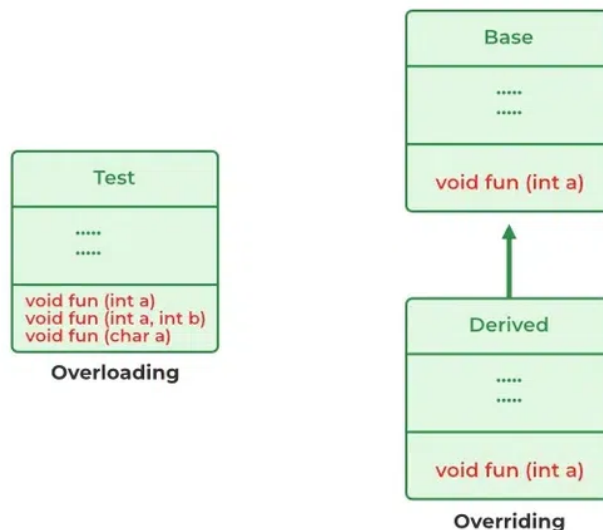
```
4 // Base Class
5 class Parent {
6     void show() { System.out.println("Parent's show()");
7 }
8
9 // Inherited class
10 class Child extends Parent {
11     // This method overrides show() of Parent
12     void show() { System.out.println("Child's show()");
13 }
14
15 // Inherited class
16 class GrandChild extends Child {
17     // This method overrides show() of Parent
18     void show()
19     {
20         System.out.println("GrandChild's show()");
21     }
22 }
23
24 // Driver class
25 class Main {
26     public static void main(String[] args)
27     {
28         Parent obj1 = new GrandChild();
29         obj1.show();
30     }
31 }
```

Output

```
GrandChild's show()
```

Method Overriding vs Method Overloading

1. [Overloading](#) is about the same method having different signatures. Overriding is about the same method, and same signature but different classes connected through inheritance.



2. Overloading is an example of compiler-time polymorphism and overriding is an example of run-time [polymorphism](#).

FAQs on Java Method Overriding

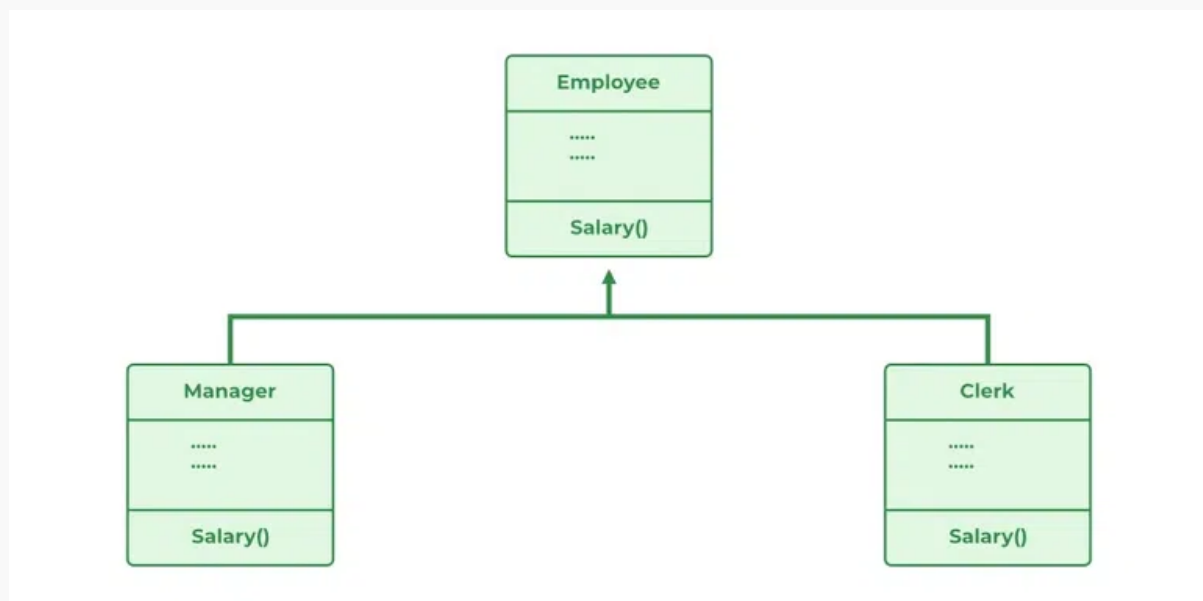
Q1. What is Method Overriding?

As stated earlier, overridden methods allow Java to support [run-time polymorphism](#). Polymorphism is essential to object-oriented programming for one reason: it allows a general class to specify methods that will be common to all of its derivatives while allowing subclasses to define the specific implementation of some or all of those methods. Overridden methods are another way that Java implements the “one interface, multiple methods” aspect of polymorphism. [Dynamic Method Dispatch](#) is one of the most powerful mechanisms that object-oriented design brings to bear on code reuse and robustness. The ability to exist code libraries to call methods on instances of new classes without recompiling while maintaining a clean abstract interface is a profoundly powerful tool. Overridden methods allow us to call methods of any of the derived classes without even knowing the type of derived class object.

Q2. When to apply Method Overriding? (with example)

Overriding and [Inheritance](#): Part of the key to successfully applying polymorphism is understanding that the superclasses and subclasses form a hierarchy that moves from lesser to greater specialization. Used correctly, the superclass provides all elements that a subclass can use directly. It also

defines those methods that the derived class must implement on its own. This allows the subclass the flexibility to define its methods, yet still enforces a consistent interface. **Thus, by combining inheritance with overridden methods, a superclass can define the general form of the methods that will be used by all of its subclasses.** Let's look at a more practical example that uses method overriding. Consider an employee management software for an organization, let the code has a simple base class *Employee*, and the class has methods like *raiseSalary()*, *transfer()*, *promote()*, .. etc. Different types of employees like *Manager*, *Engineer*, ..etc may have their implementations of the methods present in base class *Employee*. In our complete software, we just need to pass a list of employees everywhere and call appropriate methods without even knowing the type of employee. For example, we can easily raise the salary of all employees by iterating through the list of employees. Every type of employee may have its logic in its class, we don't need to worry because if *raiseSalary()* is present for a specific employee type, only that method would be called.



Java

1

```
// Java program to demonstrate application
```

```

2 // of overriding in Java
3
4 // Base Class
5 class Employee {
6     public static int base = 10000;
7     int salary() { return base; }
8 }
9
10 // Inherited class
11 class Manager extends Employee {
12     // This method overrides salary() of Parent
13     int salary() { return base + 20000; }
14 }
15
16 // Inherited class
17 class Clerk extends Employee {
18     // This method overrides salary() of Parent
19     int salary() { return base + 10000; }
20 }
21
22 // Driver class
23 class Main {
24     // This method can be used to print the salary
    of
25     // any type of employee using base class
    reference
26     static void printSalary(Employee e)
27     {
28         System.out.println(e.salary());
29     }
30
31     public static void main(String[] args)
32     {
33         Employee obj1 = new Manager();
34
35         // We could also get type of employee
    using
36         // one more overridden method. loke
    getType()
37         System.out.print("Manager's salary : ");
38         printSalary(obj1);
39

```

```
40         Employee obj2 = new Clerk();
41         System.out.print("Clerk's salary : ");
42         printSalary(obj2);
43     }
44 }
```

Output

```
Manager's salary : 30000
Clerk's salary : 20000
```

Related Article

- [Dynamic Method Dispatch or Runtime Polymorphism in Java](#)
- [Overriding equals\(\) method of Object class](#)
- [Overriding toString\(\) method of Object class](#)
- [Overloading in java](#)
- [Output of Java program | Set 18 \(Overriding\)](#)